



DLMDSEDE02 Project: Data Engineering Implementation Phase Report

Submitted By

Joyan Sharukh Bhatena

Matriculation: 9213297

Table of Contents

Table of Contents.....	2
Executive Summary.....	3
Project Architecture Overview.....	3
Technical Implementation.....	3
1. Project Structure.....	3
2. Data Ingestion Microservice.....	3
3. Data Processing Microservice.....	4
4. Data Visualization Microservice.....	5
5. Container Orchestration.....	6
Technical Achievements.....	6
Performance Optimizations.....	6
Data Quality Measures.....	6
Scalability Considerations.....	6
Challenges and Solutions.....	6
Conclusion.....	7

Kisan Call Centre Data Pipeline Implementation Report

Link to github repository: <https://github.com/Joyan9/kcc-farmer-queries-pipeline>

Executive Summary

This report documents the implementation of a batch-processing data system designed to handle farmer query data from the Kisan Call Centre (KCC), sourced from the Indian Government's Open Data Portal. The system processes approximately 9 million records of farmer queries to provide valuable insights for agricultural planning and support through a containerized microservices architecture.

Project Architecture Overview

The implemented solution follows a microservices architecture with three primary components:

- **Data Ingestion Service:** Automated data extraction from KCC API
- **Data Processing Service:** ETL operations and star schema transformation
- **Data Visualization Service:** Analytics dashboard and reporting

Technical Implementation

1. Project Structure

The project was organized with clear separation of concerns:

```
kcc-farmer-queries-pipeline/
├── docker/                # Container definitions
├── ingestion/             # Data ingestion microservice
├── processing/            # ETL and data transformation
├── visualization/         # Analytics dashboards
├── storage/               # Data storage (raw + processed)
├── docs/                  # Architecture diagrams
└── docker-compose.yml     # Orchestration configuration
```

2. Data Ingestion Microservice

Implementation Approach:

- Containerized Python application using `ingestion.Dockerfile`
- Parameterized execution using argparse for flexible operation modes

- Support for both incremental and backfill processing patterns

Key Features

- **Flexible Execution Modes:**
 - Regular runs (last month data)
 - Complete backfill operations
- **Data Format Optimization:** Transitioned from JSON to Parquet format for better compression and Spark compatibility
- **Robust API Integration:** Implemented using DLT (Data Load Tool) for built-in retry capabilities
- **Pagination Handling:** Custom while-loop implementation to handle API pagination (default 10 rows per call)

Docker Commands

```
# Build ingestion service
docker build -f docker/ingestion.Dockerfile -t kcc-ingestion .

# Run backfill (limited sample)
docker run --rm -v $(pwd)/storage:/app/storage kcc-ingestion --backfill
--max-offset 200

# Run for last month only
docker run --rm -v $(pwd)/storage:/app/storage kcc-ingestion
```

Challenges Faced & Resolved

- **Data Format Issues:** Initial JSON format caused nested field complications in Spark; resolved by adopting Parquet format for raw data files.
- **API Pagination:** Undocumented 10-row default limit required custom pagination logic
- **Performance Optimization:** Implemented 50,000 row limit per month to prevent excessive data ingestion times

3. Data Processing Microservice

Architecture

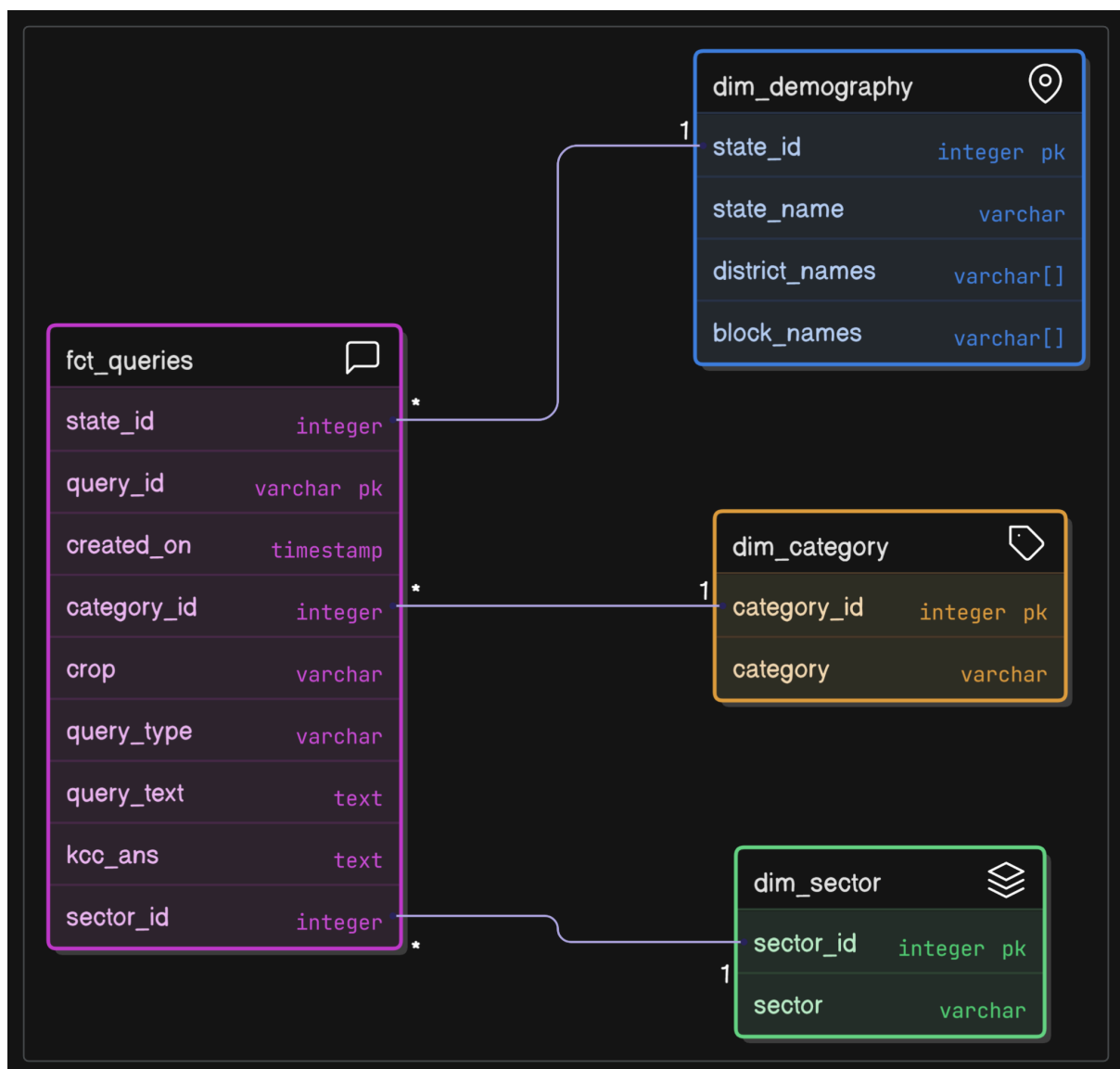
```
processing/
├── config.py           # Configuration management
├── main.py             # Main orchestration script
├── initial_load.py     # Backfill processing
├── incremental_load.py # Incremental processing
└── helpers/
    ├── cleaning.py     # Data cleaning and PII masking
    └── dimensions.py   # Dimension table generation
```

```
io.py # I/O operations
```

Data Transformation Pipeline

1. **Data Cleaning:** Null value replacement and invalid data handling
2. **PII Masking:** Regex-based personally identifiable information protection
3. **Surrogate Key Generation:** Created artificial keys for dimensional modeling
4. **Star Schema Implementation:** Fact and dimension table creation

Star Schema Design



Docker Commands

```
# Build processing service
docker build -f docker/processing.Dockerfile -t kcc-processing .
```

```
# Run initial load (backfill processing)
docker run --rm -v $(pwd)/storage:/app/storage kcc-processing python
main.py --job initial

# Run incremental processing
docker run --rm -v $(pwd)/storage:/app/storage kcc-processing python
main.py --job incremental
```

4. Data Visualization Microservice

Implementation

- Jupyter Lab-based analytics environment
- DuckDB integration for high-performance analytics queries
- Comprehensive visualization suite using matplotlib, seaborn, and plotly

Dashboard Features

- Geographic analysis (state-level insights)
- Category and sector breakdowns
- Crop-specific analytics
- State vs. category heatmaps
- Seasonal pattern analysis

Docker Commands

```
# Build visualization service

docker build -f docker/visualize.Dockerfile -t kcc-visualize .

# Launch Jupyter Lab

docker run --rm -p 8888:8888 -v $(pwd)/storage:/app/storage -v
$(pwd)/visualization/notebooks:/app/notebooks kcc-visualize
```

5. Container Orchestration

Docker Compose Configuration: Implemented orchestration with service dependencies

- **Ingestion Service:** Runs backfill process with 100-row limit per month as sample data
- **Processing Service:** Executes initial load job with Spark UI access (port 4040)
- **Visualization Service:** Provides Jupyter Lab interface (port 8888)

Service Dependencies

- Processing depends on successful ingestion completion
- Visualization depends on successful processing completion

Challenges and Solutions

Challenge	Impact	Solution
JSON Nesting Issues	Data corruption in Spark	Migrated to Parquet format
API Pagination	Limited data retrieval	Implemented custom pagination loop
Processing Performance	Long execution times	Added row limits and spark job optimization

Conclusion

The Kisan Call Centre data pipeline implementation successfully demonstrates a robust, scalable approach to processing large-scale agricultural data. The microservices architecture provides flexibility for individual component scaling and maintenance, while the containerized deployment ensures consistent environments across development and production.

The system's ability to process 9 million records with comprehensive data transformation, quality controls, and visualization capabilities positions it as a solid foundation for agricultural data analytics and farmer support initiatives.

The implementation overcame significant technical challenges around data format compatibility, API limitations, and performance optimization, resulting in a production-ready solution that can support ongoing agricultural planning and policy decisions.